Proyecto final - Fundamentos de Lenguajes Programación

Robinson Duque, Ph.D
robinson.duque@correounivalle.edu.co

Noviembre de 2026

1. Introducción

El presente proyecto tiene por objeto enfrentar a los estudiantes del curso:

- a la comprensión de varios de los conceptos vistos en clase
- a la implementación de un lenguaje de programación
- al análisis de estructuras sintácticas, de datos y de control de un lenguaje de programación para la implementación de un interpretador

Importante: Tenga en cuenta que las descripciones presentadas en este documento son generales y pretenden dar un contexto que le permita a cada estudiante entender el proyecto y las características del lenguaje que debe desarrollar. Así mismo, se proponen algunas construcciones sintácticas, **es posible que usted requiera modificar o adaptar levemente la sintaxis para cumplir con los requerimientos.**

2. LogicFusion - Lenguaje funcional e imperativo con soporte para lógica booleana

LogicFusion es un lenguaje de programación que combina características de los paradigmas funcional e imperativo. Permite definir variables y constantes, construir y manipular estructuras de datos, definir procedimientos recursivos y utilizar estructuras de control. Adicionalmente, ofrece soporte para representar y resolver problemas de satisfacibilidad booleana (SAT, por sus siglas en inglés), mediante fórmulas expresadas en forma normal conjuntiva (FNC).

Se propone para este proyecto que usted implemente un lenguaje de programación para lo cual se brindan algunas ideas de sintáxis básica. La semántica del lenguaje estará determinada por las especificaciones en este proyecto. Recuerde que usted podrá realizar leves ajustes para poder completar la implementación.

- El desarrollo del proyecto y la participación de los integrantes debe ser rastreable y verificable en todo momento (no podrán aparecer entregas de la nada o cambios completos del interpretador sin evidenciar un desarrollo contínuo). Para esto, cada grupo deberá crear un **repositorio** en **Github** con el nombre (LogicFusion). Si se le solicita, deberá proporcionar accesos al repositorio al momento de la sustentación. Recuerde incluir la información de los integrantes en el archivo Readme al igual que información relevante del proyecto).
- La gramática deberá contener ejemplos de cada producción utilizando llamados a `scan&parse`. Es decir, deberá contener ejemplos de cómo se crean las variables, procedimientos, invocación a procedimientos, etc.

2.1. Valores:

- **Valores denotados:** Ref(valores expresados)
- **Valores expresados:** enteros, flotantes, cadenas de caracteres, booleanos (True, False), procedimientos, listas, vectores, diccionarios, instancias SAT (se explicarán más adelante).

Sugerencia: trabaje los valores enteros y flotantes desde la especificación léxica. Implemente los demás valores desde la especificación gramatical.

2.2. (70 pts) Sintaxis Gramatical

2.2.1. Expresiones

En LogicFusion, casi todas las estructuras sintáticas son expresiones, y todas las expresiones producen un valor. Las expresiones pueden ser clasificadas en:

- **Identificadores:** Son secuencias de caracteres alfanuméricos que comienzan con una letra.

```
<expresion>  := <identificador>
<identificador>::=<letter>| {<letter>|
    0,...,9| }*
<letter> ::= A..Z | a..z
```

- **Definiciones:** LogicFusion se divide en dos bloques principales: GLOBALS y PROGRAM. El bloque GLOBALS contiene declaraciones globales de variables, constantes y funciones que pueden

ser accedidas desde el bloque PROGRAM. PROGRAM contiene la función principal y el código ejecutable. Ejemplo:

```
GLOBALS {
  int a = 10;
  float pi = 3.1415;
  string nombre = "Robinson";
  bool docente? = True;
  void sinTipo = 5;
  list<int> nums1 = (1, 2, 3);
  list<float> nums2 = (1.5, 2.3, 3.8);
  vector<int> nums3 = [1, 2, 3];
  vector<float> nums4 = [1.5, 2.3, 3.8];
  const int valor = 10;
  const float valor2 = 20;
  dict<string, int> edades = {"Juan": 25,
      "Ana": 30};
  dict<string, void> datos = {"Nombre": "
      Juan", "Edad": 40, "Casado": True};

  proc square = lambda (int x) {
    return x * x;
  }

} %End of Globals
PROGRAM{

  proc main() {
    return square(a);
  }
}
```

Una definición de variables introduce una colección de variables actualizables y sus valores iniciales. Una definición *const* introduce una colección de constantes no actualizables y sus valores iniciales.

- **Validación parcial de tipos de datos primitivos:** Los lenguajes de programación tipados requieren de ciertas definiciones de reglas y se basan en dos posibles enfoques de implementación (chequeo de tipos / inferencia de tipos). Sin embargo, este lenguaje de programación no es del todo un lenguaje fuertemente tipado, y sólo se realizarán validaciones de tipo en dos momentos:
  - al momento de la creación de variables se deberá verificar que el tipo de dato indicado para una variable o una constante coincida con su valor inicial asignado. En caso que no exista coincidencia, se deberá lanzar algún mensaje de error.
  - si se intenta modificar el contenido de una variable, se deberá validar que el tipo de la variable registrada en los ambientes conincida con el tipo del nuevo valor asignado. En caso que no exista coincidencia, se deberá lanzar algún mensaje de error.

  A las variables de tipo *void* no se les debe hacer validación de tipo alguna ya que permite crear variables sin tipo. Es decir, su contenido podrá cambiar de manera indistinta.

- **Tipos de datos primitivos:**
  - int: Representa un número entero. Ejemplo: int a = 10;
  - float: Número decimal de precisión simple. Ejemplo: float pi = 3.1415;
  - string: Cadena de caracteres delimitada por comillas dobles (”). Ejemplo: string name = “LogicFusion”;
  - bool: Tipo booleano con valores True o False. Ejemplo: bool isActive = True;
  - void: Tipo que permite representar variables sin tipo. Las variables definidas con este tipo de dato podrán cambiar su valor libremente durante la ejecución. Ejemplo: void sinTipo = 5;
  - *sat*: Representa una instancia del problema de satisfacibilidad booleana (SAT), expresada en forma normal conjuntiva (FNC). Ejemplo:

    ```
    sat problema = FNC 3 ((1 or -2) and
    (2));
    ```

  A los tipos de datos primitivos se les puede anteponer la palabra *const* para indicar que representará una variable inmutable. Ejemplo: const float pi = 3.1415;

- **Tipos de datos compuestos:**
  - $list < T >$: Lista inmutable, donde T representa el tipo de dato de sus elementos (int, float, string, bool). Ejemplo: $list < int >$ nums = (1, 2, 3);
  - $vector < T >$: Estructura de datos mutable de tipo T que puede cambiar sus contenidos y de tamaño. Ejemplo: $vector < float >$ values = [1.0, 2.5, 3.8];
  - $dict < T, U >$: Estructura de datos mutable que se declaran con $dict < T, U >$, donde T representa el tipo de las claves (inmutables) y U el tipo de los valores mutables. Por Ejemplo: $dict < string, int > edades = \{"Juan" : 25, "Ana" : 30\}$;

- **Funciones:**
  - Se le sugiere que las funciones **proc** las utilice como un tipo de dato más en el lenguaje.
  - **Declaración:** $proc < nombre >= lambda(< tipos >< parametros >) \{return < cuerpo >\}$

- Las funciones pueden ser definidas como valores de tipo **proc**. Por Ejemplo: $proc\ square\ =\ lambda(int\ x)\{return\ x * x;\}$.
- Las funciones por defecto deben soportar la recursión.
- El paso de parámetros será por valor.

- **Primitivas y Operaciones Básicas:**
  - **Operaciones aritméticas**: $+, -, *, /$, Soportadas para int y float.
  - **Operaciones relacionales**: $==, !=, >, <, >=, <=$, Retornan bool. Las comparaciones funcionan con int, bool y float
  - **Operaciones de asignación**: $->$, Permiten cambiar valores a variables existentes. Por ejemplo: $area -> base * height/2$;
  - **Listas**: son inmutables; no permiten modificación después de su declaración. Usted debe proponer la sintaxis que le permita representar primitivas para simular el comportamiento de: `empty?` (para validar si una lista está vacía), `empty` (para representar la lista vacía), `make-list` (para crear una lista con valores específicos), `list?` (para validar si un valor es una lista), `head` (para obtener el primer elemento de la lista), `tail` (para obtener el resto de la lista), `append` para unir dos listas (ver adendo de listas con las funciones completas), `ref-list` para referenciar una posición de la lista y `length` para consultar la longitud de la lista.
  - **Vectores:** son mutables; los valores se pueden modificar, agregar nuevos elementos en alguna posición o al final o eliminar elementos. Usted debe proponer la sintaxis que le permita representar primitivas que simulen el comportamiento de: `vector?` (para validar si un valor es un vector), `make-vector` (para crear un vector de un tamaño determinado), `ref-vector` (para obtener el valor de un vector en una posición específica), `set-vector` (para cambiar el valor de una de las posiciones de un vector). Incluir también la operación `append-vector` para agregar nuevos elementos al final del vector (los nuevos valores deben coincidir con el tipo de dato en la definición del vector). La operación `delete-val-vector[pos]` elimina el elemento de la posición *pos* y desplaza una posición a la izquierda todos los elementos a la derecha de la posición *pos*. La operación `insert-val-vector[val, pos]` inserta el valor *val* la posición *pos* y desplaza una posición a la derecha todos los elementos a la derecha de la posición *pos*.
  - **Diccionarios:** se debe extender el lenguaje y agregar manejo de diccionarios. Son estructuras de clave valor donde las claves son únicas e inmutables y los valores son mutables a través de las claves. Se deben crear primitivas que simulen el comportamiento de: `dict?` (para averiguar si un valor es un diccionario), `make-dict` (para crear un diccionario con claves y valores específicos), `ref-dict` (para obtener el valor de una de las claves del diccionario), `set-dict` (para cambiar el valor asociado a una de las claves del diccinario). Algunas operaciones comunes para los diccionarios son: `append-dict` (para agregar nuevos elementos clave y valor al diccionario); `keys-dict`: Retorna una lista de todas las claves en el diccionario; `values-dict`: Retorna una lista de todos los valores en el diccionario. Cualquier adición o modificación al diccionario debe coincidir con la definición inicial de su tipo de datos.
  - Cadenas: deben soportar las primitivas *longitud* y *concatenar*

- **Secuenciación:** La secuenciación se puede lograr con dos instrucciones:
  - **Secuencia** $BLOCK$: Permite ejecutar una secuencia de expresiones en orden y retornar el resultado de la última (debe tener mínimo una expresión). La secuencia $BLOCK$ puede ser utilizada dentro de funciones o dentro de la función principal *main*. Ejemplo:

```
GLOBALS {
  int a = 10;
  proc square = lambda (int x) {
    return BLOCK {
      print("Funcion square");
      x * x;
    }
  }

} %End of Globals
PROGRAM{

 proc main() {
   return BLOCK {
    print("Bienvenidos a LogicFusion")
        ;
    square(a);
   }
 }
}
```

  - **Secuencia** $LOCALS$: Permite crear un ambiente local, evaluar las declaraciones y ejecutar una secuencia de expresiones dentro de ese ambiente. Contiene un espacio para creación de cualquier cantidad de variables locales (también funciones) y un cuerpo donde se pueden acceder las variables y escribir

cualquier cantidad de instrucciones, al final se retorna la ejecución de la última instrucción (debe contener por lo menos una declaración local). La secuencia *LOCALS* puede ser utilizada dentro de funciones o dentro de la función principal *main*:

```
GLOBALS {
  float base = 10.5;

  proc triangle = lambda (float base,
      float height) {
    return LOCALS {
      float area = 0;
      float b = base;
      float h = height;
    } {
     print("Esta es la primera instruccion
          del cuerpo");
     print("Esta es la segunda instruccion
          y la siguiente es la tercera");
     area -> base * height / 2;
     area;
    } %end of locals returns the area
  } %end of triangle

} %End of Globals
PROGRAM{

  proc main() {
    return triangle(base, 6.4);
  }

}
```

Alcance de variables: el intento de acceder a una variable debe intentarse primero a nivel local en el bloque LOCALS. En caso que la variable que se busca no esté en ese bloque, se debe acceder al bloque anterior (en caso de anidamiento) o hasta llegar al bloque GLOBALS. Por último, si la variable no se encuentra en el bloque GLOBALS, quiere decir que la variable no existe en el programa.

- **Variables (mutables e inmutables)**: dentro del bloque GLOBALS o en un bloque LOCALS se crea una colección de variables actualizables y sus valores iniciales. Una variable mutable puede ser modificada cuantas veces se desee con el mismo tipo de dato **(un intento de asignar una variable con un tipo de dato distinto, deberá lanzar un error**), esta regla no aplica a variables tipo *void* que podrán cambiar su tipo de dato en cualquier momento. Una variable mutable puede ser declarada así: `int a = 5; bool b = False;`. En ambos casos, ambas variables podrán ser modificadas durante la ejecución de un programa. Por ejemplo: `a ->9;` o `b->True;` son asignaciones válidas. Esta asignación `a ->True ;` deberá lanzar un error ya que se está asignando un tipo de dato incorrecto. **El intento de modificar una variable inmutable (const), deberá generar un error.**

- **Iteración:** el lenguaje debe permitir la defición de estructuras de repetición tipo `while y for`. Por ejemplo:

```
GLOBALS {
  int max_num = 5;
  proc factorial = lambda (int num) {
     return LOCALS {
         int result = 1;
         int i = 1;
     } {
        for (i ->1; i <= num; i -> i+1) {
            result -> result * i;
         }
         result; %Retorna el factorial
     } %End of locals
   } %End factorial
} %End globals

PROGRAM {
    proc main() {
       return  LOCALS {
            int count = 1;
            int fact  = 0;
        } {
            while (count <= max_num) {
                print("Cuenta: ", count);
                count -> count + 1;
            }

            fact -> factorial(5);
            print("El factorial de 5 es:
                ", fact);
        } %End of locals
    } %end of main
} %end of Program
```

.

- **Estructuras de control**

  Proponga e implemente en su lenguaje de programación las estructuras de control **if** y **switch**.

2.3. (30 pts) Soporte para instancias SAT

El problema de satisfactibilidad booleana (SAT, por sus siglas en inglés) consiste en determinar si existe una asignación de valores de verdad para un conjunto de variables booleanas que permita satisfacer una fórmula lógica. En este proyecto se trabajará con fórmulas expresadas en forma normal conjuntiva (FNC).

Una fórmula en FNC está compuesta por una conjunción de cláusulas, donde cada cláusula contiene una disyunción de literales. Un literal es una variable booleana o su negación.

Por ejemplo, considere la siguiente fórmula:

$$C = (x \vee \neg y \vee z \vee w) \wedge (\neg y \vee z) \wedge (\neg x \vee \neg y \vee \neg z) \wedge (z \vee w) \wedge y$$

El problema consiste en determinar si existe una asignación de valores para las variables (x, y, z, w) que haga verdadera la fórmula C.

Para este proyecto se implementará un mecanismo de búsqueda exhaustiva que explore las posibles asignaciones de valores booleanos. No se requiere implementar algoritmos avanzados de resolución SAT. El procedimiento podrá detener la búsqueda tan pronto encuentre una asignación satisfactoria. Si se exploran todas las asignaciones posibles y ninguna satisface la fórmula, se deberá concluir que la instancia es insatisfactible.

Representación de instancias SAT

Las instancias SAT deben ser valores expresados del lenguaje LogicFusion. Por tanto, deberán poder utilizarse en cualquier contexto donde se permita una expresión, por ejemplo, como valor inicial de una variable, argumento de un procedimiento o valor de retorno.

Para representar las fórmulas se utilizará una codificación numérica de las variables y sus negaciones. Una variable se representa mediante un entero positivo y su negación mediante el mismo entero precedido por el signo menos.

Por ejemplo, para la fórmula anterior se tiene la siguiente correspondencia:

$$x = 1, \quad y = 2, \quad z = 3, \quad w = 4$$

De esta manera, la fórmula puede representarse así:

```
FNC 4 (
    (1 or -2 or 3 or 4) and
    (-2 or 3) and
    (-1 or -2 or -3) and
    (3 or 4) and
    (2)
)
```

La palabra reservada `FNC` indica que se está construyendo una instancia SAT. El número que aparece a continuación indica la cantidad de variables booleanas de la instancia. Cada número representa un literal: un entero positivo representa una variable y uno negativo representa su negación.

La gramática deberá incorporar esta construcción como una expresión del lenguaje. Conceptualmente, se propone la siguiente gramática:

```
<expresion> ::= <instancia-sat>

<instancia-sat> ::= FNC <entero>
                    ( <clausula-sat>
                      (and <clausula-sat>)* )

<clausula-sat> ::= ( <literal-sat>
                     (or <literal-sat>)* )

<literal-sat> ::= <entero>
              ::= -<entero>
```

La notación anterior describe la estructura esperada. Cada grupo deberá adaptarla a las convenciones de SLLGEN y a la especificación léxica que utilice en su interpretador.

Se deben cumplir las siguientes condiciones:

- El número de variables de la instancia debe ser un entero positivo.
- Los índices de las variables deben estar entre 1 y n, donde n es el número de variables declarado en la instancia.
- Una cláusula contiene uno o más literales.
- Las cláusulas se combinan mediante el operador `and`, mientras que los literales de una cláusula se combinan mediante el operador `or`.
- Una misma variable puede aparecer varias veces, incluso con signos diferentes.

Evaluación de instancias SAT

Se deberá implementar una operación primitiva denominada `solve`, que reciba una instancia SAT y realice su evaluación mediante búsqueda exhaustiva.

La operación tendrá la siguiente forma:

```
solve(<instancia-sat>)
```

El resultado de la operación será un valor tipo *void* que indique si la instancia es satisfactible o insatisfactible. En caso de encontrar una solución, deberá retornar también una asignación de valores booleanos para todas las variables de la instancia.

La asignación se representará mediante una lista de valores booleanos, donde la primera posición corresponde a la variable 1, la segunda a la variable 2 y así sucesivamente.

Por ejemplo, una posible solución para la instancia presentada anteriormente es:

```
(False True True True)
```

Esto significa que x es falso, y es verdadero, z es verdadero y w es verdadero.

Considere los siguientes ejemplos de uso dentro de LogicFusion:

Ejemplo 1: instancia satisfactible

```
GLOBALS {
    sat x = FNC 4 (
        (1 or -2 or 3 or 4) and
        (-2 or 3) and
        (-1 or -2 or -3) and
        (3 or 4) and
        (2)
    );
} %End globals

PROGRAM {
    proc main() {
        return solve(x);
    } %End of main
} %End of Program
```

La ejecución de `solve(x)` deberá retornar un resultado equivalente a:

```
(satisfactible (False True True True))
```

La asignación presentada es una de las posibles soluciones. No es necesario encontrar todas las asignaciones que satisfacen la fórmula.

Ejemplo 2: instancia insatisfactible

```
GLOBALS {
    sat x = FNC 2 (
        (1 or 2) and
        (-1) and
        (-2)
    );
} %End globals

PROGRAM {
    proc main() {
        return solve(x);
    } %End of main
} %End of Program
```

En este caso, la fórmula equivale a:

$$C = (x \vee y) \wedge (\neg x) \wedge (\neg y)$$

No existe una asignación de valores para x e y que satisfaga simultáneamente todas las cláusulas. Por tanto, la ejecución deberá retornar:

```
(insatisfactible ())
```

Consideraciones de implementación

- Las instancias SAT deberán representarse mediante una estructura de datos propia dentro del interpretador. No deberán tratarse simplemente como cadenas de caracteres.
- La operación `solve` deberá recibir un valor de tipo instancia SAT. Si recibe un valor de otro tipo, deberá generar un error.
- La evaluación deberá explorar las 2^n posibles asignaciones de las n variables, o detenerse antes si encuentra una asignación satisfactoria.
- Cada cláusula será verdadera si al menos uno de sus literales es verdadero. La instancia completa será verdadera si todas sus cláusulas son verdaderas.
- Si se encuentra una asignación satisfactoria, el resultado deberá contener dicha asignación. Si no se encuentra ninguna, deberá retornar un resultado de insatisfactibilidad.
- La evaluación de una instancia SAT no deberá modificar las variables, listas, vectores o diccionarios definidos en el programa.

3. Evaluación

El proyecto debe ser realizado en los grupos que se definan para este propósito y utilizando la librería SLLGEN de Dr Racket. Este debe ser sustentado y cada persona del grupo obtendrá una nota entre 0 y 1 (por sustentación), la cual se multiplicará por la nota obtenida en el proyecto (la sustentación se llevará a cabo en el campus virtual). El interpretador podrá contener algunas variaciones simples del lenguaje base. Dado el caso que un grupo se presente con un lenguaje distinto a la gramática definida para el interpretador (o generado completamente con IA o sin justificación del uso de estructuras/optimizaciones avanzadas en interpretadores o compiladores), obtendrán una nota de 0 en la sustentación asociada con dicho interpretador.

4. Responsabilidad y cumplimiento del trabajo grupal

El líder de cada grupo será responsable de hacer seguimiento al cumplimiento de las actividades y entregas por parte de sus integrantes, de acuerdo con los plazos y acuerdos establecidos internamente por el grupo. Asimismo, deberá informar oportunamente al docente cuando algún integrante no esté participando o no esté cumpliendo con las responsabilidades asignadas (esto se hará a través de un correo dirigido al docente con copia a todos los integrantes). El grupo deberá definir internamente cómo se asumirán y distribuirán las actividades pendientes para garantizar el cumplimiento de las entregas.

No se aceptarán entregas individuales de estudiantes que no hayan participado oportunamente en la elaboración del trabajo, **ni entregas realizadas por estudiantes en grupos diferentes** a los establecidos en clase. La participación y los aportes de cada integrante deberán corresponder al trabajo desarrollado por el grupo.

5. Adendo - Listas

Las **listas** en **LogicFusion** son estructuras de datos inmutables que almacenan elementos de un mismo tipo (salvo que se definan de tipo *void*). Permiten acceder a sus elementos, obtener sublistas y construir nuevas listas mediante operaciones que no modifican la estructura original. Se declaran mediante el tipo `list<T>`, donde T representa el tipo de sus elementos.

5.0.1. `empty`

Representa una lista vacía.

```
list<int> nums = empty;
```

5.0.2. `empty?(lst)`

Retorna `True` si la lista está vacía y `False` en caso contrario.

```
empty?(empty);        # True
empty?([1, 2, 3]);    # False
```

5.0.3. `make-list(elem, lst)`

Construye una nueva lista agregando el elemento `elem` al inicio de la lista `lst`. Ambos deben tener elementos del mismo tipo.

```
list<int> nums = make-list(1, empty);
nums = make-list(2, nums);
# Resultado: (2 1)
```

5.0.4. `list?(x)`

Retorna `True` si el valor `x` es una lista y `False` en caso contrario.

```
list?(empty);         # True
list?(42);            # False
```

5.0.5. `head(lst)`

Retorna el primer elemento de la lista. Genera un error si la lista está vacía.

```
list<int> nums = (10, 20, 30);

head(nums);    # 10
```

5.0.6. `tail(lst)`

Retorna una nueva lista con todos los elementos, excepto el primero. Genera un error si la lista está vacía.

```
list<int> nums = (10, 20, 30);

tail(nums);    # (20 30)
```

5.0.7. `append(lst1, lst2)`

Concatena dos listas y retorna una nueva lista con los elementos de ambas. Las listas deben tener el mismo tipo de elementos.

```
list<int> a = (1, 2);
list<int> b = (3, 4);

append(a, b);    # (1 2 3 4)
```

5.0.8. `ref-list(lst, pos)`

Retorna el elemento almacenado en la posición `pos` de la lista. Los índices comienzan en cero. Genera un error si la posición está fuera de rango.

```
list<int> nums = (10, 20, 30);

ref-list(nums, 1);    # 20
```

5.0.9. `length(lst)`

Retorna la cantidad de elementos almacenados en la lista.

```
list<int> nums = (10, 20, 30);

length(nums);    # 3
```

5.0.10. Nota semántica

Las listas son estructuras inmutables. Por tanto, las operaciones `make-list`, `tail` y `append` generan nuevas listas sin modificar las originales.

Una lista puede representarse conceptualmente mediante una estructura enlazada de elementos:

$$lst = (e_1, e_2, \ldots, e_n)$$

La operación `head(lst)` retorna el primer elemento, mientras que `tail(lst)` retorna el resto de la lista. La operación `append(lst1, lst2)` construye una nueva lista que contiene los elementos de ambas listas.

6. Adendo - Diccionarios

Los **diccionarios** en **LogicFusion** son estructuras que almacenan pares `(clave, valor)`. Cada clave es única e inmutable, mientras que los valores asociados pueden modificarse. Se utilizan para representar información mediante asociaciones entre claves y valores.

6.0.1. `create-dict()`

Crea un nuevo diccionario vacío.

```
dict<string,int> d = create-dict();
```

6.0.2. `dict?(x)`

Retorna `True` si el valor `x` es un diccionario y `False` en caso contrario.

```
dict<string,int> d = create-dict();

dict?(d);   # True
dict?(42);  # False
```

6.0.3. `ref-dict(dic, clave)`

Retorna el valor asociado a la `clave` dentro del diccionario `dic`. Si la clave no existe, genera un error.

```
dict<string,int> d = create-dict();
set-dict(d, "edad", 34);

ref-dict(d, "edad");   # 34
```

6.0.4. `set-dict(dic, clave, valor)`

Inserta un nuevo par clave-valor o actualiza el valor asociado a una clave existente. Esta operación modifica el diccionario.

```
dict<string,int> d = create-dict();

set-dict(d, "edad", 34);
set-dict(d, "edad", 35);

ref-dict(d, "edad");   # 35
```

6.0.5. `keys(dic)`

Retorna una lista con todas las claves almacenadas en el diccionario. El orden de los elementos no está garantizado.

```
keys(d);
```

6.0.6. `values(dic)`

Retorna una lista con todos los valores almacenados en el diccionario, conservando el orden correspondiente a las claves.

```
values(d);
```

6.0.7. Nota semántica

Un diccionario puede representarse internamente como una colección de pares `(clave, valor)`:

$$dic = \{(k_1, v_1), (k_2, v_2), \ldots, (k_n, v_n)\}$$

La operación `ref-dict(dic, clave)` busca la clave y retorna el valor asociado. Si no se encuentra, genera un error.

Por su parte, `set-dict(dic, clave, valor)` inserta un nuevo par si la clave no existe o reemplaza el valor asociado si ya está presente. Las claves deben ser únicas y los tipos de las claves y los valores deben respetar la declaración `dict<T,U>`.

7. Adendo - Vectores

Los **vectores** en **LogicFusion** son estructuras de datos mutables que almacenan elementos de un mismo tipo. A diferencia de las listas, permiten modificar sus elementos, agregar nuevos valores al final y eliminar elementos de posiciones específicas. Se declaran mediante el tipo `vector<T>`, donde T representa el tipo de sus elementos.

7.0.1. `vector?(x)`

Retorna `True` si el valor `x` es un vector y `False` en caso contrario.

```
vector<int> v = [10, 20, 30];

vector?(v);   # True
vector?(42);  # False
```

7.0.2. `make-vector(n)`

Crea un vector de tamaño `n`. La inicialización de sus posiciones debe respetar el tipo de dato declarado.

```
vector<int> v = make-vector(3);
```

7.0.3. `ref-vector(vec, pos)`

Retorna el elemento almacenado en la posición `pos` del vector. Genera un error si la posición está fuera de rango.

```
vector<int> v = [10, 20, 30];

ref-vector(v, 1);   # 20
```

7.0.4. `set-vector(vec, pos, valor)`

Modifica el elemento ubicado en la posición `pos` por el nuevo `valor`. El valor asignado debe coincidir con el tipo de dato del vector.

```
vector<int> v = [10, 20, 30];

set-vector(v, 1, 50);
ref-vector(v, 1);   # 50
```

7.0.5. `append-vector(vec, valor)`

Agrega un nuevo elemento al final del vector. El nuevo valor debe coincidir con el tipo de dato declarado.

```
vector<int> v = [10, 20, 30];

append-vector(v, 40);
# Resultado: [10, 20, 30, 40]
```

7.0.6. `delete-val-vector(vec, pos)`

Elimina el elemento ubicado en la posición `pos`. Los elementos que se encuentran a su derecha se desplazan una posición hacia la izquierda.

```
vector<int> v = [10, 20, 30, 40];

delete-val-vector(v, 1);
# Resultado: [10, 30, 40]
```

7.0.7. `insert-val-vector[vec, pos, val]`

Inserta el valor `val` en la posición `pos` y desplaza una posición a la derecha todos los elementos a la derecha de la posición `pos`.

```
vector<int> v = [10, 20, 30, 40];

insert-val-vector(v, 1, -5);
# Resultado: [10, -5, 20, 30, 40]
```

7.0.8. Nota semántica

Los vectores son estructuras mutables y permiten modificar su contenido y tamaño durante la ejecución del programa. Las operaciones de acceso, modificación y eliminación utilizan índices que comienzan en cero. Las operaciones `set-vector`, `append-vector` `insert-val-vector` y `delete-val-vector` modifican directamente el vector recibido.

Todas las operaciones deben garantizar que los elementos almacenados respeten el tipo declarado en `vector<T>`, excepto cuando se utiliza el tipo de dato *void*.

8. Adendo - Primitivas de Control y Recursión

Las **estructuras de control** en **LogicFusion** permiten modificar el flujo de ejecución de los programas mediante decisiones y ciclos. Por su parte, las **funciones recursivas** permiten resolver problemas mediante procedimientos que se invocan a sí mismos.

Estas construcciones se integran con el modelo de evaluación del lenguaje. Las expresiones condicionales y los ciclos evalúan sus bloques de acuerdo con las condiciones establecidas, mientras que los procedimientos pueden recibir parámetros y retornar resultados mediante la instrucción `return`.

8.0.1. `if ... else`

Evalúa una expresión condicional y ejecuta uno de dos bloques, dependiendo del resultado de su evaluación.

En LogicFusion, las condiciones utilizan representación numérica: el valor `0` se interpreta como falso y cualquier otro valor como verdadero. La expresión condicional debe tener un resultado numérico.

```
int edad = 20;
...
...
if (edad >= 18) {
    "Mayor de edad"
} else {
    "Menor de edad"
}
```

También es posible anidar estructuras condicionales para evaluar múltiples alternativas:

```
int edad = 15;
...
...
if (edad < 13) {
    "ñNio"
} else {
    if (edad < 18) {
        "Adolescente"
    } else {
        "Adulto"
    }
}
```

Cada rama contiene un bloque de expresiones. La evaluación de la estructura retorna el resultado del bloque correspondiente a la condición seleccionada.

8.0.2. `switch`

Permite seleccionar una alternativa entre varias opciones, comparando el resultado de una expresión con los valores especificados en las cláusulas `case`. Debe incluir una cláusula `default`, que se ejecuta cuando ninguno de los casos coincide.

```
string color = "rojo";
...
...

switch color {
    case "rojo": {
        "Detente"
    }
    case "amarillo": {
        "Precaucion"
    }
    case "verde": {
        "Sigue"
    }
    default: {
        "Color desconocido"
    }
}
```

La expresión evaluada se compara con los valores de cada cláusula `case`. Se ejecuta únicamente el bloque correspondiente a la primera coincidencia; si no existe ninguna, se ejecuta el bloque `default`.

8.0.3. `while (condicion) { ... }`

Ejecuta repetidamente un bloque de expresiones mientras la condición especificada sea verdadera. La condición se escribe entre paréntesis y el cuerpo del ciclo se delimita mediante llaves.

```
int contador = 0;

while (contador < 5) {
    print("Iteracion: ", contador);
    contador -> contador + 1;
}
```

La condición se evalúa antes de cada iteración. Si el resultado de la condición es diferente de cero (i.e., True), se ejecuta el bloque; si es igual a cero (i.e., False), el ciclo termina.

El ciclo permite modificar variables dentro de su cuerpo, por lo que es responsabilidad del programador garantizar que la condición eventualmente deje de cumplirse, evitando ciclos infinitos.

8.0.4. `for (inicializacion; condicion; actualizacion) { ... }`

Permite ejecutar repetidamente un bloque de expresiones utilizando una estructura de control que integra tres componentes:

- **Inicialización:** establece el valor inicial de la variable de control.
- **Condición:** determina si se ejecuta una nueva iteración.
- **Actualización:** modifica la variable de control al finalizar cada iteración.

Los tres componentes se escriben entre paréntesis y se separan mediante punto y coma. Las asignaciones utilizan el operador `->` de LogicFusion.

```
LOCALS {
    int resultado = 1;
    int i = 1;
} {
    for (i -> 1; i <= 5; i -> i + 1) {
        resultado -> resultado * i;
    }
    resultado;
}
```

En este ejemplo, la variable `i` se inicializa en uno, el ciclo se ejecuta mientras su valor sea menor o igual a cinco y, al finalizar cada iteración, se incrementa en uno.

El orden de ejecución de sus componentes es el siguiente:

1. Se ejecuta la inicialización una sola vez, antes de comenzar el ciclo.
2. Se evalúa la condición. Si el resultado de la condición es diferente de cero (i.e., True), se ejecuta el cuerpo del ciclo; si es cero (i.e., False), el ciclo termina.
3. Se ejecuta la actualización de la variable de control.
4. Se vuelve al paso de evaluación de la condición.

Si la condición es falsa desde la primera evaluación, el cuerpo del ciclo no se ejecuta.

8.0.5. `proc ... = lambda (...) { ... return ... }`

Los procedimientos permiten encapsular expresiones y operaciones que pueden reutilizarse mediante llamadas. Se definen mediante la palabra reservada `proc`, un identificador, la palabra `lambda` y una lista de parámetros tipados.

```
proc cuadrado = lambda (int x) {
    return x * x;
}
```

Los procedimientos pueden recibir cero o más parámetros y retornar valores mediante la instrucción `return`. El valor retornado debe ser compatible con el tipo de resultado esperado por el contexto en el que se utiliza.

Una llamada a un procedimiento se realiza utilizando su identificador y suministrando los argumentos correspondientes:

```
int resultado = cuadrado(4);
```

Los procedimientos pueden utilizar otros procedimientos y realizar llamadas a sí mismos. Su definición genera una clausura que conserva los parámetros, el cuerpo y el ambiente declarativo correspondiente.

8.0.6. Funciones recursivas

Un procedimiento es recursivo cuando realiza una llamada a sí mismo durante su ejecución. Esta estrategia permite expresar soluciones para problemas cuya definición puede descomponerse en instancias más pequeñas del mismo problema.

Una función recursiva debe contar con:

- **Caso base:** condición que permite finalizar las llamadas recursivas.
- **Caso recursivo:** operación que realiza una nueva llamada al procedimiento con argumentos que aproximan el cálculo al caso base.

Por ejemplo, el cálculo del factorial de un número puede definirse recursivamente:

```
proc factorial = lambda (int n) {
    if (n <= 1) {
        return 1;
    } else {
        return (n * factorial(n - 1));
    }
}
...
...
int resultado = factorial(5);
```

8.0.7. Consideraciones semánticas

- Las condiciones de las estructuras de control deben producir valores numéricos. El valor `0` representa falso y cualquier otro valor representa verdadero.
- Las estructuras condicionales ejecutan únicamente el bloque correspondiente a la condición evaluada.
- Los procedimientos pueden retornar valores de los tipos admitidos por LogicFusion.
- Las llamadas recursivas generan nuevas ejecuciones del procedimiento, conservando los argumentos y el contexto de evaluación correspondiente.
- La ejecución de un procedimiento recursivo debe alcanzar un caso base para evitar llamadas indefinidas.